AI ENGINEER INTERVIEW · BLUE HARBOUR

How I build software

Jessie Tarrosa · senior full-stack engineer in Cebu, 9+ years shipping production software, building with AI agents every day

jessiet.dev · github.com/oreoseenoevil · October 2026

01 · JOURNEY

From web design to owning products

2015

Web design intern

My first web role, at a real-estate company.

2020

Freelance full-stack

Building web apps for clients, front end to database.

2021

Accenture

Workforce analytics, skill graphs and MLOps tooling for a 15,000-person organization.

2022

Senior full-stack, CoDev

Fintech wallet and rewards flows, then a records platform with AI-assisted review.

2025

Founder

My own company. Atrium OS, built AI-first since June 2026, runs 3 pilot clients on contract.

The thread through all of it: I like owning the whole path, from the data model to the deploy, and AI agents let one engineer do that at a much larger scale.

02 · CODING WITH AI

What AI changed about how I code

6,000+
commits on Atrium OS

4.5 mo
from first commit to the October release

1
engineer, working with AI agents

1. The work moved from typing to specifying and reviewing. Plans and written rules come first.
2. Agents run in parallel, but each owns its own files, or they overwrite each other.
3. The model matches the risk: the strongest one for money, permissions, privacy and concurrency; cheaper ones for search and QA.
4. Agents never touch production data, and I read every diff before it lands.
5. Decisions get written down with a date, so the next agent session inherits them instead of re-arguing them.

03 · WORKFLOW

How a change gets built

1 · Plan

I write the spec and decide the design. Rules live in AGENTS.md.

2 · Build

Agents work in parallel, each owning its own files.

3 · Gate

Lint, both typechecks and the build on every push. Risky code gets tests.

4 · Review

I read every diff and try the feature in the app.

5 · Release

Staging first, index check, backup, then production.

Tools: Claude Code every day, with parallel subagents for implementation, review and QA.

Guardrails: scripts default to a dry run, and no agent touches production data.

04 · STACK

One typed language, end to end

Layer	Choice	Why
Front end	Next.js 16, React 19, Tailwind 4	One app for admin, resident and supplier portals.
API	tRPC 11 with Zod 4	A breaking API change fails the compile, not production.
Data	MongoDB with transactions	Ledger and payment writes commit together or not at all.
Tests	node:test, Playwright, axe	Real transactions in every test; accessibility checked.
Ops	Docker, GHCR, Caddy	Cheap and simple to run, with staging first.

TypeScript everywhere matters more with agents: the compiler catches what a fast-moving agent misses. Also in production elsewhere: PostgreSQL, Supabase and MySQL.

05 · QUALITY AND TESTING

Tests go where a bug costs money

14,312
unit tests, all passing in the last full run

8,874 / 8,875
integration tests passing, each on a real database

28 / 28
recent pushes green on lint, typecheck and build

- Money, ledger, permissions and privacy get an integration test while they’re built. The rest catches up in the release pass.
- Each integration file boots its own in-memory MongoDB replica set, so transactions behave like production. Suites run in 12 shards.
- Honest gap: the full suite runs locally before releases. Making it green in CI on every push is next.

06 · SECURITY

Access is checked at the API, not in the UI

Layered checks

Every call needs an active property and a membership, keeps portal accounts out, then checks the module and action.

Tenant scoping, tested

Every query carries the organization. A role-parity test checks each role against a separate permission matrix.

Safe operations

Scripts default to a dry run. Nightly backups, a weekly restore check, and agents never touch production data.

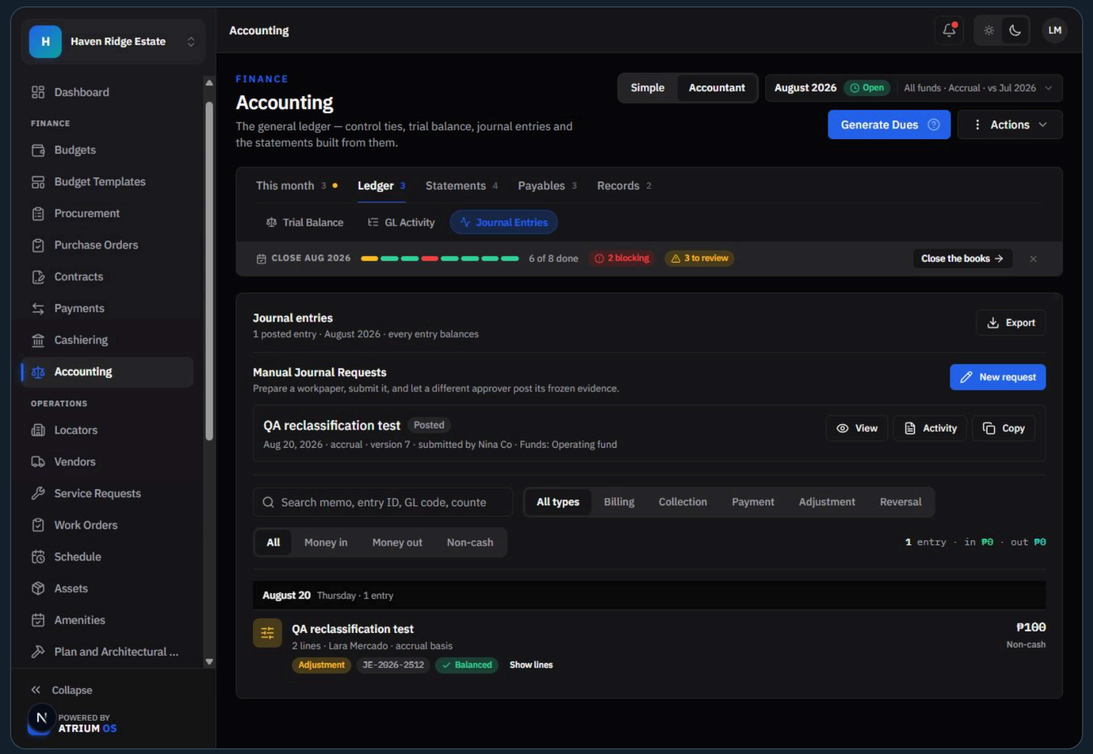
Gaps I’m closing

The Content Security Policy still runs in report-only mode, and a dependency audit isn’t in Atrium’s CI yet.

07 · WORK I'M PROUD OF

Money that posts exactly once

- Payment and journal entries commit in one database transaction.
- Every payment carries an idempotency key, unique per organization. A retry replays the first result, a changed request under a used key is refused, and integrity faults fail closed.
- A build-time test scans the source and fails if anyone, human or agent, adds a payment writer outside that path.
- Every 15 minutes a sweep re-checks pending provider checkouts through the same claim a late callback uses.



Atrium OS ledger, demo data: a manual journal posted only after a second person approved it.

07 · WORK I'M PROUD OF

How I knew it worked

22

Tests on the rules

Integration tests cover the idempotency registry that every payment goes through.

1

Dry run on real data

Before the October release, the reconcile script ran read-only on production and found 1 line to fix.

0

Apply, then prove zero

After the fix was applied, the final dry run reported 0 lines to change.

Belt and braces: payments refuse to run if a critical unique index is missing, and the release checklist confirmed all 79 unique indexes before anything shipped.

The honest answer on evaluation

I haven’t yet shipped an LLM feature to end users with a formal evaluation behind it. Here’s the closest real thing, and how I’d evaluate one for your clients.

Running daily: my job-search agent

- Twice a day it reads job boards, scores posts, drafts applications and logs each run.
- 196 jobs tracked over 15 logged runs. It surfaced this role.
- Guardrails: posts are untrusted input, claims come only from a fact file, it never signs in or pays, and a tested tool guards my tracker.
- Evaluation today is outcomes, not an eval set. That’s the gap I’d close first.

How I’d evaluate one for a client

1. Agree the task and what a wrong answer costs.
2. Build a golden set from real, consented examples.
3. Automatic checks plus a human rubric; a model grader only once it agrees with people.
4. Run evals in CI on every prompt or model change, and block regressions.
5. Ship behind a flag, sample outputs for human review, watch cost and latency.

09 · PUBLIC CODE

Code you can read today

oreoseenoevil.github.io

My portfolio site

React and TypeScript. Tests and lint run in a pre-commit hook, and CLAUDE.md and AGENTS.md show the rules my agents work under.

crewboard

A Workers pane for Claude Code

My fork of claude-kit (MIT, credited in the README): progress for subagents, local and cloud workers and long tasks, with 18 tests.

Atrium OS and my client work stay private, so I've shown them here through screens, results and design notes instead of a repo.

What I’d bring as your first engineer

Ownership

Schema to deploy, including the unglamorous parts: backups, staging and releases.

Fast, human-gated delivery

Agents for speed, written rules, tests where mistakes cost money, and every diff reviewed.

Evals before features

An evaluation loop in place before any LLM feature reaches your clients.

Thank you · jessiet.dev · github.com/oreoseenoevil